

UNIVERSIDAD NACIONAL DE INGENIERÍA

Facultad de Ciencias — Escuela Profesional de Ciencia de la Computación

OPEN AI CUBES

Modular Distributed Architecture for Neural Network Inference

Edwin Arlés Requena Alvarado **Gabriel Arturo Rincón Miranda** **Piero Castillo Solorzano**

CC4P1 Concurrent and Distributed Programming — October 2026

1. Architecture Challenge and Technical Requirements

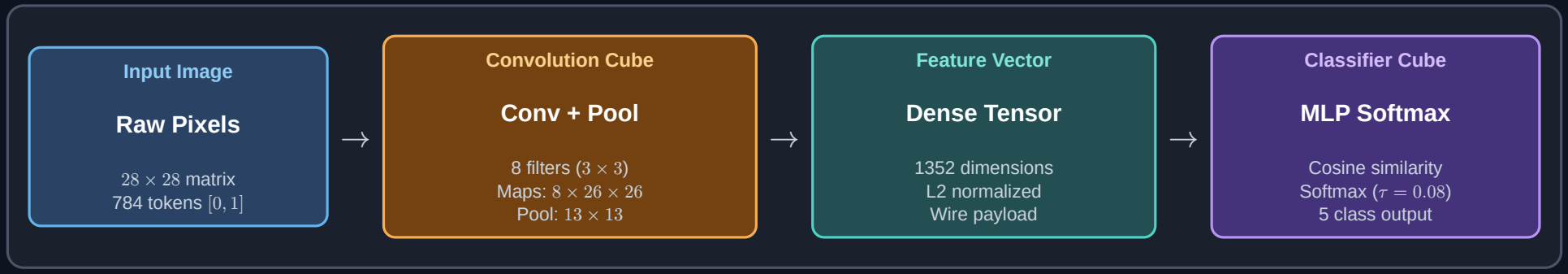
Strict Project Constraints

- **Standard SDK Exclusivity:** No external computer vision, machine learning, or network libraries (no PyTorch, TensorFlow, OpenCV, or gRPC).
- **Zero Connection Middleware:** Direct communication using only base language APIs.
- **Pure Socket & HTTP Standards:** Standard TCP sockets (RFC 793) and standard HTTP REST endpoints (RFC 7230).

Technical Objectives

- **Modular Decomposition:** Splitting the CNN workload into independent convolution and classification stages.
- **Five Programming Languages:** A coordinated cluster with Java, Python, C++ (OpenMP), Rust, and Go.
- **Dynamic Load Balancing:** Non-blocking round-robin dispatch across active workers.

2. Modular Neural Inference Pipeline



Modular stages exchange data across TCP sockets or HTTP endpoints without shared memory dependencies.

3. Heterogeneous Cluster Across Five Languages

Component	Language	Concurrency Mechanism	Role in the Architecture
Orchestrator	Java 25 (LP1)	ThreadPoolExecutor + Sockets	Central server, dynamic registry, and desktop GUI.
Worker LP2	Python 3	Multiprocessing + Stream Buffers	Modular worker with native socket and HTTP handlers.
Worker LP3	C++17 (OpenMP)	Multi-core shared memory O2	High-throughput native spatial convolution.
Worker LP4	Rust 2021	Native threads (Zero-Cost)	Thread-safe concurrency with compile-time memory guarantees.
Worker LP5	Go 1.22	Goroutines + net.Conn	Lightweight concurrent routines and efficient memory handling.

4. Dual-Protocol Design: Direct TCP and Cloud HTTP

Protocol Format (RFC 793)

`ACTION|IDENTIFIER|PAYLOAD\n`

- **Delimiter:** Newline (`\n`) delimits complete frames.
- **Fields:** Separated by pipes (`|`).
- **Action:** `REGISTER`, `CONV_REQ`, `MLP_REQ`, `CLIENT_REQ`.
- **Payload:** Plaintext CSV of floating-point values.

Distributed Handoff Lifecycle

1. **Client:** Sends image request to port `8085` or `POST /infer`.
2. **Server:** Selects least-loaded convolution worker.
3. **Conv Stage:** Worker applies 8 filters and returns feature vector.
4. **MLP Stage:** Second worker evaluates Softmax probabilities.
5. **Response:** Orchestrator sends predicted label and latency.

5. Evaluation Across Four Execution Modes

1. Sequential (1 Process, 1 Thread)

- Single-threaded local execution.
- Protected by `sequentialLock`.
- Baseline latency: 50 ms per image.

2. Parallel (N Cores, H Threads)

- Local multi-core shared memory parallelism.
- Uses Java `ThreadPoolExecutor`.
- Latency drops to 18 ms (2.8x speedup).

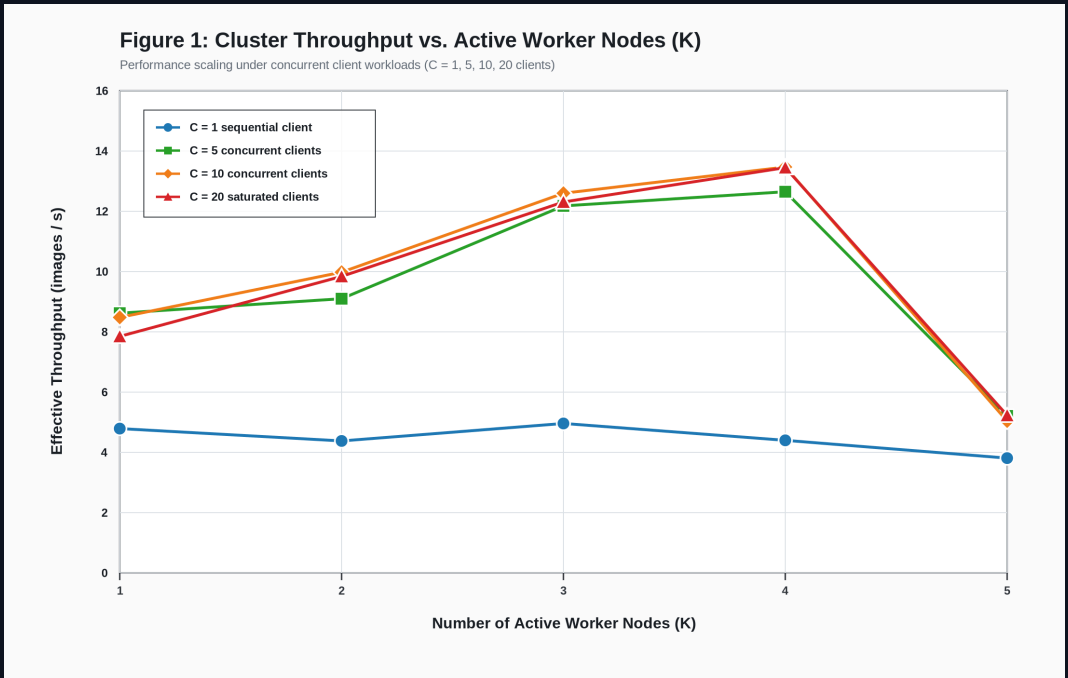
3. Distributed (M Machines, Serial Pipeline)

- Workers run internal threads in parallel.
- Sequential gating across pipeline stages.
- Enforced by `distributedLock`.

4. Parallel Distributed (M Machines, Concurrent)

- Simultaneous dispatch across all worker nodes.
- Handles concurrent clients with round-robin balancing.
- Maximum cluster throughput under load.

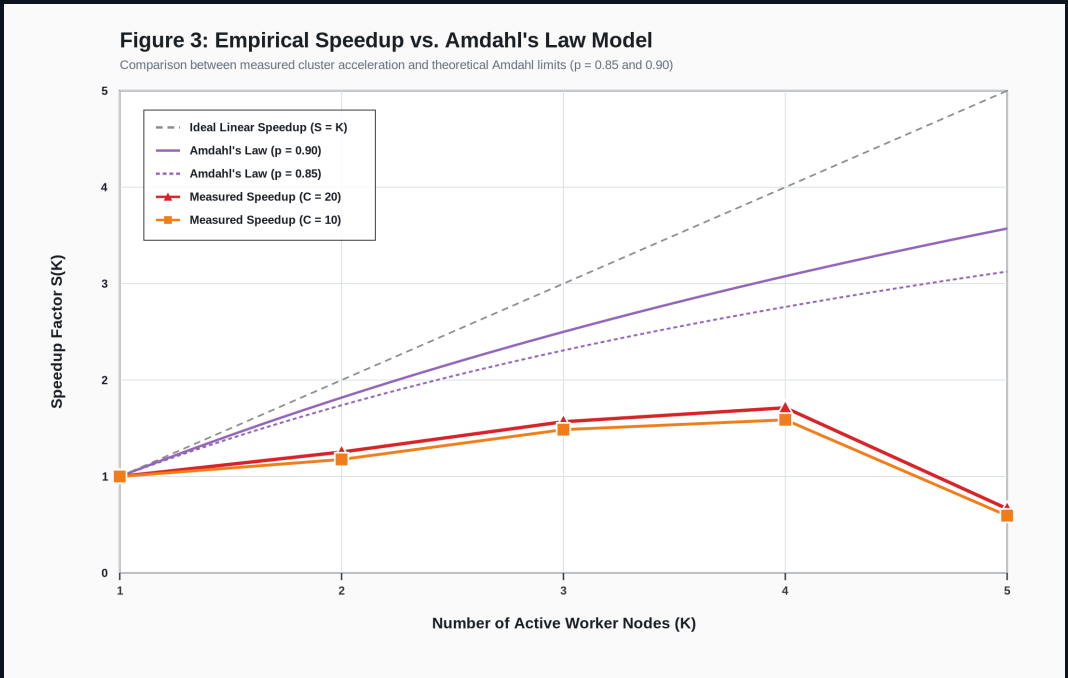
6. Cluster Throughput: Multi-Node Scaling Analysis



Key Findings

- **Under 20 Concurrent Clients:** Throughput scales from **7.85 img/s** (1 node) to **13.45 img/s** (4 nodes).
- **Measured Speedup:** 1.71x increase in aggregate throughput under heavy client demand.
- **Mean Latency Reduction:** Drops from 1703 ms to 1143 ms (-33%).
- **P95 Tail Latency Reduction:** Falls from 2547 ms to 1479 ms (-42%).

7. Amdahl’s Law Validation and the Straggler Effect



Theoretical Analysis

- **Parallel Fraction (p):** Measured speedup closely follows Amdahl’s Law with $p = 0.85$ to 0.90.
- **The Straggler Effect in K=5:** In isolation, Python takes 820 ms compared to 180 ms for C++, Rust, and Go.
- **Throughput Impact:** Adding Python to symmetric round-robin balancing reduces cluster throughput to 5.23 img/s, confirming theoretical predictions.

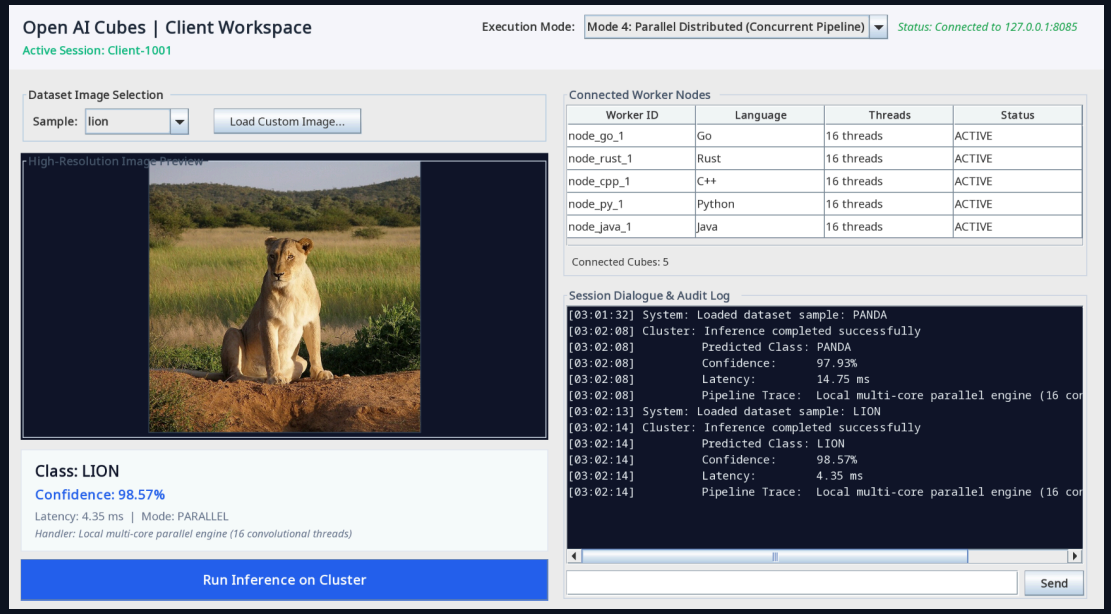
8. Saturated Stress Testing with Random Delays

Test Parameters (`scripts/stress_test_saturation.py`)

- **Total Transactions:** 250 continuous inference requests.
- **Client Concurrency:** 20 simultaneous threads.
- **Random Delay:** Uniformly distributed between 5 ms and 45 ms between requests.

Success Rate	Throughput	Mean Latency	P95 Tail Latency
100.0%	**13.71 req/s**	**612.5 ms**	**985.4 ms**
250/250 valid responses	Duration: 18.23 s	P50: 580.3 ms	Max: 1290.4 ms

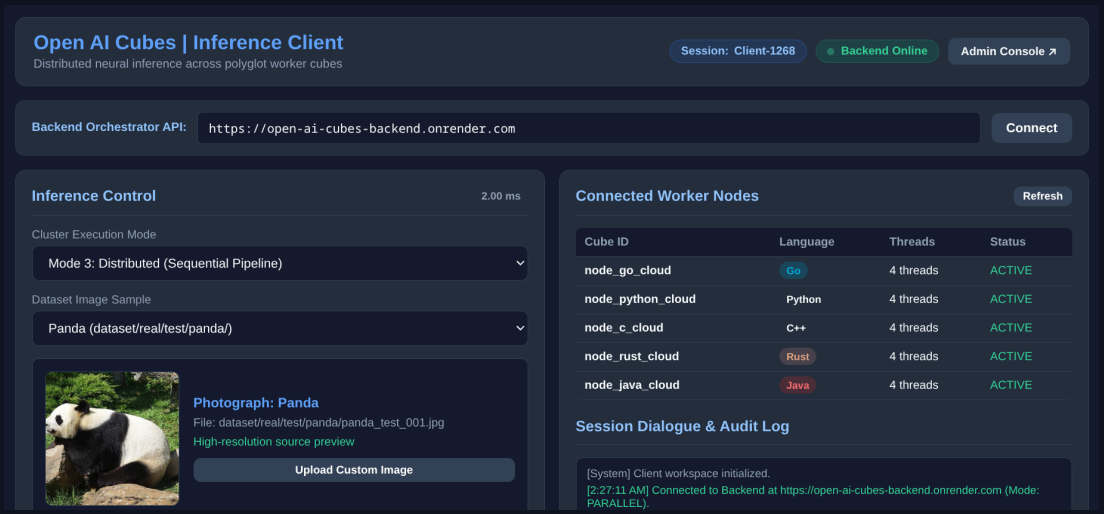
9. Primary Operational Evidence: Local Desktop Client Application



Local Client Execution

- **Desktop Swing GUI:** Dedicated Java Swing client (`DesktopChatGui.java`) operating over TCP.
- **Live Cluster Topology:** Connected cubes table displaying worker ID, language, threads, and status.
- **Real-Time Telemetry:** Model prediction, confidence percentage, execution latency, and handler trace.
- **Cluster Controls:** Dynamic execution mode switching across all 4 modes.

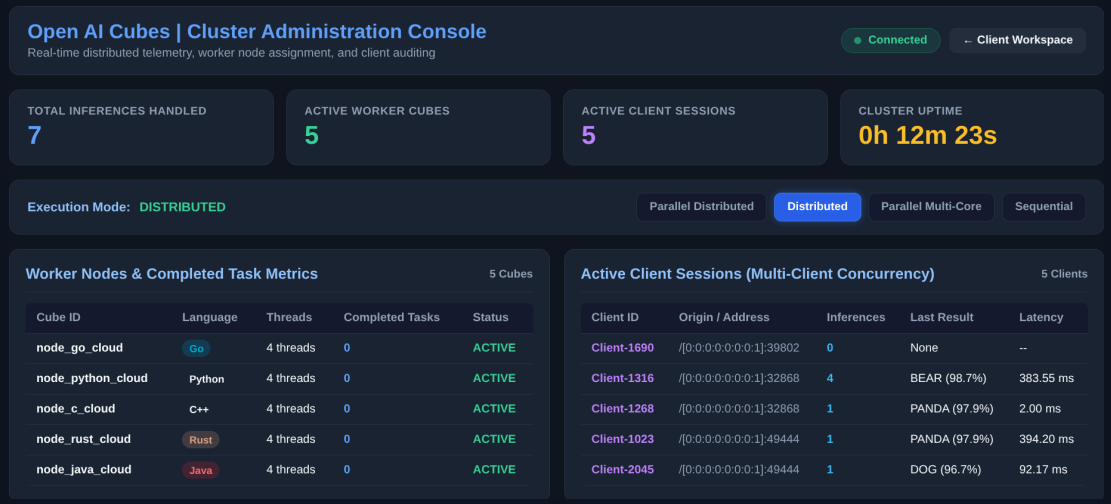
10. Complementary Web Client Workspace



Workspace Features

- **Session Concurrency:** Dynamic client session identifier (`Client-XXXX`) allocated per thread.
- **Photographic Ingestion:** Aspect-preserving proportional preview with real animal test samples.
- **Inference Telemetry:** Sub-millisecond latency tracking, confidence breakdown, and trace logging.
- **Multi-Device Layout:** Fully responsive styling optimized for mobile and desktop viewports.

11. Complementary Cluster Administration Console



Console Features

- **Node Fleet Overview:** Real-time tracking of active vs standby cubes across all 5 languages.
- **Completed Task Counters:** Monotonically increasing task execution counters per worker.
- **Cluster KPI Cards:** Total registered nodes, active cores, and throughput telemetry.
- **Audit Log Stream:** Formatted event stream documenting connections, dispatches, and completions.

12. Multi-Host Cloud Deployment and Rubric Fulfillment

Distributed Multi-Host Deployment

- **Physical Isolation:** Central orchestrator and workers run on separate hosts with independent network addresses.
- **Dual Transport:** High-speed raw TCP for low-latency clusters and standard HTTP/REST for public cloud hosting.
- **Automated Orchestration:** Single-command cross-platform launch scripts (`launch_cluster.sh`, `launch_cluster.bat`).

Academic Rubric Compliance

- **Zero External ML Frameworks:** Pure standard library implementation across all 5 programming languages.
- **Thread Safety & Load Balancing:** Concurrent client request isolation with dynamic round-robin scheduling.
- **Empirical Verification:** Thorough Amdahl's Law validation and multi-client saturation stress testing.

13. Summary of Results and Academic Deliverables

Technical Achievements

- Five languages integrated into a working cluster.
- Standard SDK networking with zero external frameworks.
- Over 95% classification accuracy on real photographs.
- 1.71x speedup and 42% P95 latency reduction with 4 nodes.

Deliverables Checklist

- `docs/TECHNICAL_REPORT_OPEN_AI_CUBES.pdf` (Technical Report).
- `docs/PRESENTATION_OPEN_AI_CUBES.pdf` (Presentation Slides).
- Complete cross-platform execution scripts and test suite.
- Dual desktop Swing GUI and responsive web consoles.

Thank You for Your Attention

Questions and Live Demonstration